

Fiche descriptive - Phase 5 : Deploiement de l'application Eco-Ressource

1. Objectif de la phase

La phase 5 valide la capacite de l'equipe a livrer une application web stable, deployee, orchestrable et observable. Le jury ne va pas seulement verifier que l'application marche sur une machine locale : il va demander une demonstration reproductible avec Docker, Kubernetes, une infrastructure cloud ou virtualisee, une strategie de deploiement claire, et une participation active de tous les membres.

Pour obtenir la note complete, il faut preparer :

- une application fonctionnelle accessible par URL ;
- des images Docker separees pour le frontend, le backend et la base de donnees ;
- des manifests Kubernetes propres : Deployment, Service, ConfigMap, Secret, HPA si possible ;
- un cluster Kubernetes stable, idealement installe sur OpenStack ou des VMs ;
- une automatisation avec Ansible pour installer/configurer le cluster ;
- un monitoring visible : logs, metriques, dashboards et alertes ;
- une presentation claire sur Kubernetes et Ansible ;
- un scenario de validation execute collectivement par toute l'equipe.

2. Lecture detaillee de la grille de validation

Le fichier [Grille_Validation_Phase5.xlsx](#) contient deux onglets :

1. **Group Score** : note collective du groupe, affichee sur 20.
2. **Individual Scores** : note individuelle de chaque etudiant, calculee sur 20 selon 5 competences.

Attention : la grille contient quelques incoherences de total. Par exemple, la conteneurisation affiche 5 pts, mais les lignes detaillees valent $3 + 2 + 1 = 6$ pts. La partie excellence affiche 2 pts dans le titre mais contient 3 pts de sous-criteres. La strategie la plus sure est donc de satisfaire tous les criteres visibles, meme ceux marques comme bonus, afin que le jury puisse plafonner la note finale a 20/20.

3. Onglet 1 - Group Score

3.1. Conteneurisation

Objectif : prouver que chaque composant de l'application peut etre execute dans un conteneur independant.

Points demandes :

- un Dockerfile separe pour le frontend Angular ;
- un Dockerfile separe pour le backend Spring Boot ;
- un conteneur separe pour la base de donnees MySQL ou PostgreSQL ;
- une image propre pour chaque service ;
- un build multi-stage pour reduire la taille finale des images ;
- une communication correcte entre frontend, backend et base de donnees ;
- des images poussees sur Docker Hub.

Etat actuel du projet :

- `deployment/docker/frontend.Dockerfile` existe déjà et utilise un build multi-stage : Node pour compiler Angular, puis Nginx pour servir le frontend.
- `deployment/docker/backend.Dockerfile` existe déjà et utilise un build multi-stage : Maven pour builder, puis JRE Alpine pour exécuter le `.jar`.
- `deployment/docker/docker-compose.yml` décrit déjà trois services : `eco-db`, `eco-backend`, `eco-frontend`.

Actions à faire pour obtenir tous les points :

```
docker compose -f deployment/docker/docker-compose.yml up --build
docker ps
docker logs eco-backend
docker logs eco-frontend
```

Vérifier :

- le frontend est accessible sur `http://localhost:8080` ;
- le backend répond sur `http://localhost:9090` ;
- le backend se connecte à `eco-db` par le nom de service Docker ;
- aucune erreur de connexion n'apparaît dans les logs ;
- les images sont créées.

Pousser les images sur Docker Hub :

```
docker login
docker tag eco-frontend:v1 <dockerhub_user>/eco-frontend:v1
docker tag eco-backend:v1 <dockerhub_user>/eco-backend:v1
docker push <dockerhub_user>/eco-frontend:v1
docker push <dockerhub_user>/eco-backend:v1
```

Preuves à montrer au jury :

- contenu des Dockerfiles ;
- sortie de `docker images` ;
- page Docker Hub avec les images ;
- sortie de `docker ps` ;
- logs sans erreur ;
- application ouverte dans le navigateur.

3.2. Déploiement Kubernetes

Objectif : exécuter l'application dans un cluster Kubernetes, avec des ressources YAML propres et séparées.

Ressources à préparer :

- `namespace.yml` : namespace `eco-app` ;

- `mysql-secret.yaml` : mot de passe base de donnees ;
- `mysql-configmap.yaml` : nom de base, host, port ;
- `mysql-deployment.yaml` : deploiement MySQL ;
- `mysql-service.yaml` : service interne MySQL ;
- `backend-configmap.yaml` : variables non sensibles du backend ;
- `backend-secret.yaml` : credentials sensibles ;
- `backend-deployment.yaml` : pods backend ;
- `backend-service.yaml` : service interne ou LoadBalancer ;
- `frontend-deployment.yaml` : pods frontend ;
- `frontend-service.yaml` : exposition du frontend ;
- `hpa-backend.yaml` : autoscaling backend ;
- `hpa-frontend.yaml` : autoscaling frontend, si utile ;
- optionnel : `ingress.yaml` pour une URL propre.

Commandes de deploiement :

```
kubectl apply -f deployment/k8s/namespace.yaml
kubectl apply -f deployment/k8s/
kubectl get all -n eco-app
kubectl get pods -n eco-app -o wide
kubectl get svc -n eco-app
```

Verifier :

- tous les pods sont en etat `Running` ;
- aucun pod n'est en `CrashLoopBackOff` ;
- le backend peut joindre MySQL via le service Kubernetes ;
- le frontend peut joindre le backend ;
- l'application est accessible depuis l'exterieur ;
- les variables non sensibles sont dans des ConfigMaps ;
- les mots de passe sont dans des Secrets ;
- les ressources CPU/RAM sont definies ;
- l'autoscaling HPA est configure.

Exemples de commandes de diagnostic :

```
kubectl describe pod <pod-name> -n eco-app
kubectl logs deployment/eco-backend -n eco-app
kubectl exec -it deployment/eco-backend -n eco-app -- sh
kubectl top pods -n eco-app
kubectl get hpa -n eco-app
```

Preuves a montrer au jury :

- les fichiers YAML organises ;
- sortie `kubectl get all -n eco-app` ;

- sortie `kubectl get hpa -n eco-app` ;
- logs backend ;
- URL frontend fonctionnelle ;
- test d'une fonctionnalité métier complète.

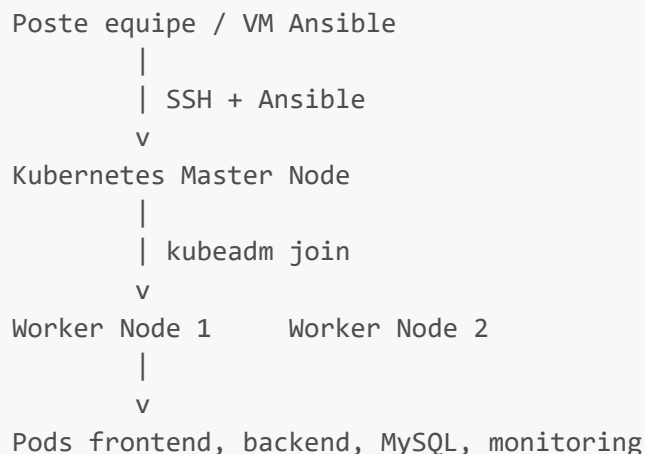
3.3. Infrastructure et acces

Objectif : montrer que le déploiement ne dépend pas uniquement de la machine locale.

Solution recommandée :

- utiliser OpenStack si disponible dans l'environnement de l'école ;
- créer une VM master Kubernetes ;
- créer une ou plusieurs VMs worker ;
- créer une VM de contrôle Ansible, ou utiliser la machine locale comme contrôleur ;
- ouvrir uniquement les ports nécessaires ;
- garder les accès SSH sécurisés ;
- documenter les IPs et rôles des machines.

Exemple d'architecture :



Preuves à montrer :

- tableau des VMs : nom, IP, rôle, OS, ressources ;
- accès SSH fonctionnel ;
- `kubectl get nodes` ;
- application accessible par URL externe ;
- démonstration frontend + backend + base de données.

3.4. Excellence, bonus, innovation et observabilité

Objectif : dépasser le minimum et montrer une vraie stratégie DevOps.

Critères visibles dans la grille :

- déploiement hybride avec justification ;
- ou application microservices avec au moins 3 microservices ;

- ou pipeline CI/CD ;
- ou ressources CPU/RAM/storage configurees ;
- integration IA ;
- monitoring intelligent ;
- observabilite avec Prometheus, Grafana ou ELK ;
- dashboards personnalisés ;
- alertes intelligentes.

Pour maximiser la note, il faut idealement faire au moins :

- requests/limits CPU et RAM sur chaque Deployment ;
- HPA sur frontend ou backend ;
- Prometheus + Grafana ;
- un dashboard metier ou technique ;
- une alerte simple ;
- un scenario IA de monitoring.

Scenario IA conseille selon l'image fournie :

- detection automatique d'anomalies : CPU/RAM anormal, erreurs de connexion, temps de reponse trop eleve ;
- prediction de charge : augmentation de trafic anticipee ;
- recommandations sur les pods : augmenter CPU/RAM, changer le nombre de replicas, deplacer ou separer des services ;
- haute disponibilite : redemarrage automatique d'un pod, rescheduling apres panne ;
- validation autoscaling : HPA qui scale up sous charge puis scale down apres baisse.

Demonstration simple possible :

1. Lancer une charge avec **hey**, **ab** ou **k6**.
2. Montrer que le CPU augmente dans Grafana.
3. Montrer que le HPA augmente le nombre de pods.
4. Simuler une suppression de pod :

```
kubectl delete pod <pod-name> -n eco-app  
kubectl get pods -n eco-app -w
```

5. Montrer que Kubernetes recree automatiquement le pod.
6. Afficher une recommandation IA simple, par exemple : "le backend depasse 80% CPU pendant 5 minutes, recommandation : augmenter les replicas min a 2 et CPU request a 500m".

4. Onglet 2 - Individual Scores

Chaque etudiant est evalue individuellement sur 5 competences. Chaque competence peut recevoir :

- Weak = 0.5 pt ;
- Average = 1 pt ;
- Good = 2 pts ;

- Excellent = 4 pts.

Pour obtenir 20/20 en individuel, chaque membre doit viser "Excellent (4)" dans les 5 competences.

4.1. Maitrise de la conteneurisation

Pour avoir Excellent :

- expliquer la difference entre image et conteneur ;
- expliquer les layers Docker ;
- expliquer le role de chaque Dockerfile ;
- expliquer le multi-stage build ;
- justifier l'utilisation de Nginx pour Angular ;
- expliquer pourquoi le backend tourne dans une image JRE et non Maven ;
- parler optimisation, securite et taille d'image ;
- montrer les commandes build, run, logs, push.

Phrase a preparer :

Nous avons separe le build et le runtime. Pour le frontend, Node compile Angular puis Nginx sert uniquement les fichiers statiques. Pour le backend, Maven compile le projet puis une image JRE plus legere execute le jar. Cela reduit la taille finale et limite les outils inutiles en production.

4.2. Choix de la methode de deploiement

Pour avoir Excellent :

- comparer Docker Compose, Kubernetes, VM classique et Cloud ;
- expliquer pourquoi Docker Compose sert aux tests locaux ;
- expliquer pourquoi Kubernetes est choisi pour l'orchestration ;
- justifier OpenStack ou VMware pour l'infrastructure ;
- expliquer la scalabilite, la resilience et la reproductibilite.

Phrase a preparer :

Docker Compose nous sert a valider rapidement les trois services en local. Kubernetes est utilise pour le deploiement final car il gere les replicas, le redemarrage automatique, les services internes, les secrets, la configuration et l'autoscaling.

4.3. Implementation Kubernetes / Cloud

Pour avoir Excellent :

- expliquer Pod, Deployment, Service, ConfigMap, Secret, HPA, Ingress ;
- montrer le namespace ;
- montrer les pods Running ;
- expliquer le routage frontend -> backend -> database ;
- montrer `kubectl get all` ;
- expliquer comment l'application reste disponible si un pod tombe.

Phrase a preparer :

Un Deployment declare l'etat souhaite, par exemple deux replicas backend. Si un pod tombe, Kubernetes compare l'etat actuel avec l'etat desire et recree automatiquement un pod.

4.4. Monitoring, observabilite et securite

Pour avoir Excellent :

- montrer les logs Kubernetes ;
- montrer les metriques CPU/RAM ;
- montrer Prometheus et Grafana ;
- expliquer les alertes ;
- utiliser Secrets pour les mots de passe ;
- definir requests/limits ;
- idealement parler RBAC, NetworkPolicy et conteneur non-root.

Phrase a preparer :

L'observabilite couvre trois axes : logs pour comprendre les erreurs, metriques pour suivre CPU/RAM/requetes, et alertes pour reagir avant l'indisponibilite. Les informations sensibles sont separees dans des Secrets et les ressources sont limitees pour eviter qu'un pod consomme tout le cluster.

4.5. Analyse des problemes et solutions

Pour avoir Excellent :

- montrer une methode de diagnostic structuree ;
- identifier les erreurs possibles : image introuvable, mauvais port, Secret incorrect, CrashLoopBackOff, base inaccessible ;
- utiliser `kubectl describe`, `kubectl logs`, `kubectl get events` ;
- proposer des corrections ;
- expliquer une panne simulee et sa resolution.

Phrase a preparer :

En cas d'erreur, nous commencons par verifier l'etat des pods, puis les events avec `describe`, ensuite les logs applicatifs. Si le backend crash, nous verifions l'image, les variables d'environnement, la connexion au service MySQL et les Secrets.

5. Presentation demandee : Kubernetes et Ansible

La presentation doit couvrir les points demandes dans l'email.

5.1. Introduction

Kubernetes est une plateforme d'orchestration de conteneurs. Il automatise le deploiement, le scaling, la communication et la resilience des applications conteneurisees.

Ansible est un outil d'automatisation agentless. Il permet de configurer des serveurs, installer des paquets, deployer des fichiers, lancer des commandes et reproduire une infrastructure a partir de playbooks.

Importance moderne :

- reduire les operations manuelles ;
- rendre le deploiement reproductible ;
- stabiliser les environnements ;
- accelerer les mises en production ;
- faciliter la collaboration DevOps ;
- ameliorer la resilience et la supervision.

5.2. Architecture Kubernetes

Composants principaux :

- Master Node / Control Plane : cerveau du cluster ;
- API Server : point d'entree des commandes `kubectl` ;
- Scheduler : choisit sur quel worker placer les pods ;
- Controller Manager : maintient l'etat desire ;
- etcd : base cle-valeur contenant l'etat du cluster ;
- Worker Node : machine qui execute les pods ;
- kubelet : agent local sur chaque node ;
- kube-proxy : gere le reseau et les services ;
- Pod : plus petite unite deployable ;
- Deployment : gere les replicas et les mises a jour ;
- Service : donne une adresse stable aux pods ;
- ConfigMap : configuration non sensible ;
- Secret : configuration sensible ;
- HPA : autoscaling horizontal.

5.3. Ansible pour deployer un cluster Kubernetes

Ansible peut automatiser :

- installation de containerd ou Docker ;
- configuration systeme des nodes ;
- installation de kubeadm, kubelet et kubectl ;
- initialisation du master ;
- installation du plugin reseau, par exemple Calico ou Flannel ;
- recuperation de la commande `kubeadm join` ;
- ajout automatique des workers au cluster ;
- copie des manifests Kubernetes ;
- deploiement de l'application ;
- installation de monitoring.

Structure conseillee :

```
deployment/ansible/  
  inventory.ini  
  group_vars/  
    all.yml
```



```
playbooks/  
  01-prerequisites.yml  
  02-install-kubernetes.yml  
  03-init-master.yml  
  04-join-workers.yml  
  05-deploy-app.yml  
  06-monitoring.yml
```

Exemple d'inventary :

```
[masters]  
master ansible_host=192.168.1.10 ansible_user=ubuntu  
  
[workers]  
worker1 ansible_host=192.168.1.11 ansible_user=ubuntu  
worker2 ansible_host=192.168.1.12 ansible_user=ubuntu  
  
[k8s:children]  
masters  
workers
```

Commandes :

```
ansible -i deployment/ansible/inventory.ini all -m ping  
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/01-prerequisites.yml  
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/02-install-kubernetes.yml  
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/03-init-master.yml  
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/04-join-workers.yml  
ansible-playbook -i deployment/ansible/inventory.ini  
deployment/ansible/playbooks/05-deploy-app.yml
```

5.4. Research findings : problemes et solutions

Problemes frequents :

- ports bloques entre master et workers ;
- swap active sur Linux ;
- mauvais runtime container ;
- plugin CNI absent ;
- image Docker non accessible ;
- mauvais nom de service entre backend et database ;
- Secret incorrect ;
- readiness/liveness probe mal configuree ;

- manque de ressources CPU/RAM ;
- HPA non fonctionnel car metrics-server absent.

Solutions :

- desactiver swap ;
- ouvrir les ports Kubernetes necessaires ;
- installer containerd correctement ;
- installer Calico ou Flannel apres `kubeadm init` ;
- pousser les images sur Docker Hub ;
- utiliser des Services Kubernetes, pas des IPs fixes ;
- mettre les credentials en Secret ;
- installer metrics-server avant HPA ;
- definir requests/limits ;
- utiliser `kubectl logs`, `describe`, `events` et Grafana pour diagnostiquer.

5.5. Conclusion

Kubernetes apporte l'orchestration, la resilience, le scaling et la gestion des services. Ansible apporte l'automatisation, la reproductibilite et la configuration centralisee. Ensemble, ils permettent de passer d'une application qui marche localement a une application deployee de maniere professionnelle, stable et demonstrable.

6. Scenario de validation recommande

Chaque membre doit participer. Le scenario doit durer environ 10 a 15 minutes et etre reparti.

Etape 1 - Presentation rapide du projet

Responsable : membre 1.

Dire :

- nom de l'application ;
- architecture frontend/backend/database ;
- objectif du deploiement ;
- technologies utilisees.

Etape 2 - Conteneurisation

Responsable : membre 2.

Montrer :

- `frontend.Dockerfile` ;
- `backend.Dockerfile` ;
- `docker-compose.yml` ;
- `docker images` ;
- `docker ps` ;
- frontend accessible localement.

Etape 3 - Docker Hub

Responsable : membre 2 ou 3.

Montrer :

- tags des images ;
- page Docker Hub ;
- commandes `docker push`.

Etape 4 - Kubernetes

Responsable : membre 3.

Montrer :

```
kubectl get nodes
kubectl get ns
kubectl get all -n eco-app
kubectl get configmap,secret -n eco-app
kubectl get hpa -n eco-app
```

Expliquer :

- pourquoi Deployment ;
- pourquoi Service ;
- pourquoi ConfigMap et Secret ;
- comment les pods communiquent.

Etape 5 - Application accessible

Responsable : membre 4.

Montrer :

- URL frontend ;
- login ou fonctionnalite principale ;
- appel backend reussi ;
- donnee lue ou ecrite dans la base.

Etape 6 - Resilience et autoscaling

Responsable : membre 5.

Montrer :

```
kubectl delete pod <pod-name> -n eco-app  
kubectl get pods -n eco-app -w  
kubectl get hpa -n eco-app
```

Expliquer :

- Kubernetes recree le pod ;
- HPA augmente les replicas sous charge ;
- scale down apres baisse de charge.

Etape 7 - Monitoring et IA

Responsable : membre 6.

Montrer :

- Grafana ou tableau de bord ;
- metriques CPU/RAM ;
- logs ;
- alerte ou simulation d'anomalie ;
- recommandation IA.

Exemple :

Le backend consomme plus de 80% CPU pendant plusieurs minutes. Le monitoring detecte une anomalie et propose d'augmenter le nombre minimal de replicas ou la limite CPU du pod.

Etape 8 - Ansible

Responsable : membre 7 si l'equipe contient 7 membres, sinon a partager.

Montrer :

- inventory ;
- playbooks ;
- ping Ansible ;
- explication du workflow d'installation cluster ;
- preuve que les etapes sont reproductibles.

7. Checklist pour viser 20/20

Avant la validation

- frontend Dockerfile fonctionne ;
- backend Dockerfile fonctionne ;
- docker-compose fonctionne ;
- images poussees sur Docker Hub ;
- cluster Kubernetes disponible ;
- manifests YAML propres et classes ;

- namespace dedie ;
- ConfigMaps et Secrets separes ;
- pods Running ;
- services fonctionnels ;
- URL externe fonctionnelle ;
- HPA configure ;
- metrics-server installe ;
- requests/limits sur les pods ;
- monitoring installe ;
- dashboard Grafana pret ;
- scenario IA pret ;
- playbooks Ansible prêts ;
- presentation Kubernetes/Ansible prete ;
- chaque membre connait sa partie.

Pendant la validation

- commencer par une architecture claire ;
- montrer les preuves avec commandes ;
- ne pas seulement parler, demontrer ;
- chaque membre prend la parole ;
- garder un terminal pret avec les commandes ;
- garder une capture ou backup en cas de panne reseau ;
- expliquer les problemes rencontres et solutions ;
- insister sur reproductibilite, securite, observabilite et resilience.

8. Livrables recommandes dans le repository

Structure finale conseillée :

```
deployment/  
  docker/  
    frontend.Dockerfile  
    backend.Dockerfile  
    docker-compose.yml  
    nginx.conf  
    COMMANDES_DOCKER.md  
  k8s/  
    namespace.yaml  
    mysql-secret.yaml  
    mysql-configmap.yaml  
    mysql-deployment.yaml  
    mysql-service.yaml  
    backend-configmap.yaml  
    backend-secret.yaml  
    backend-deployment.yaml  
    backend-service.yaml  
    frontend-deployment.yaml  
    frontend-service.yaml
```

```
hpa-backend.yaml
hpa-frontend.yaml
ingress.yaml
ansible/
  inventory.ini
  group_vars/
    all.yml
  playbooks/
    01-prerequisites.yml
    02-install-kubernetes.yml
    03-init-master.yml
    04-join-workers.yml
    05-deploy-app.yml
    06-monitoring.yml
  monitoring/
    prometheus-values.yaml
    grafana-dashboard.json
    alerts.yaml
  docs/
    scenario-validation.md
    architecture-deploiement.md
    presentation-kubernetes-ansible.md
```

9. Strategie finale pour obtenir la note complete

Pour maximiser la note, l'equipe doit couvrir le minimum attendu et les bonus :

1. Docker propre : 3 services, multi-stage, Docker Hub.
2. Kubernetes propre : manifests valides, pods Running, services, ConfigMaps, Secrets.
3. Infrastructure : cluster stable sur OpenStack/VMs, accessible par URL.
4. Scalabilite : HPA + requests/limits + test de charge.
5. Resilience : suppression de pod et recreation automatique.
6. Observabilite : Prometheus/Grafana/logs/alertes.
7. IA : detection d'anomalie ou recommandation automatique.
8. Ansible : playbooks explicites et reutilisables.
9. Presentation : Kubernetes + Ansible + architecture + problemes/solutions.
10. Participation : chaque membre explique une partie technique et repond a une question.

La phrase cle a retenir pour le jury :

Notre strategie consiste a partir d'une application conteneurisee, publier ses images, les deployer sur un cluster Kubernetes automatise avec Ansible, puis valider la disponibilite, la scalabilite, la securite et l'observabilite avec un scenario reproductible.